

Task Decomposition & Planning Algorithms

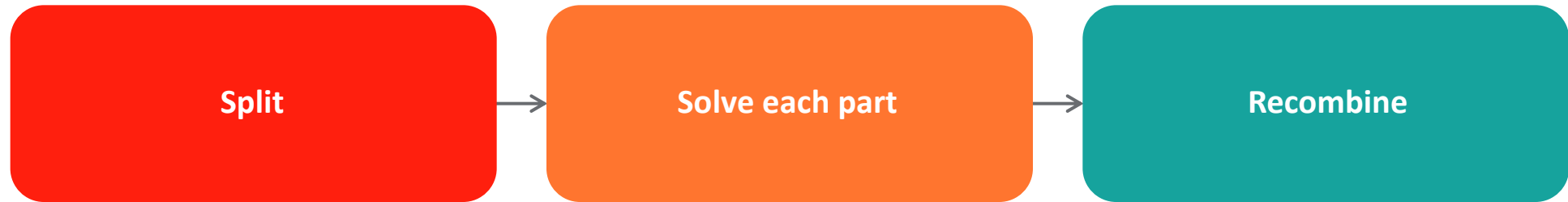
Autonomous Agents - Week 4

Why decompose?

ONE GOAL IS NOT ONE STEP

- Large goals are ambiguous, multi-step, and difficult to verify in one call.
- A monolithic prompt hides dependencies and makes failures hard to isolate.
- Smaller sub-tasks become checkable LLM calls or tool invocations.

Decomposition is divide-and-conquer for agents



Turn one intractable objective into smaller, inspectable sub-problems.

A useful planner performs two distinct jobs

1 Decompose

Break the objective into sub-goals with clear outputs.

2 Sub-plan

Choose the model call or tool, required inputs, and execution order for each sub-goal.

Keeping the jobs separate makes the plan easier to inspect, schedule, and evaluate.

Dependencies turn a task list into an executable graph

Nodes

Sub-tasks: id, instruction, required inputs, and output.

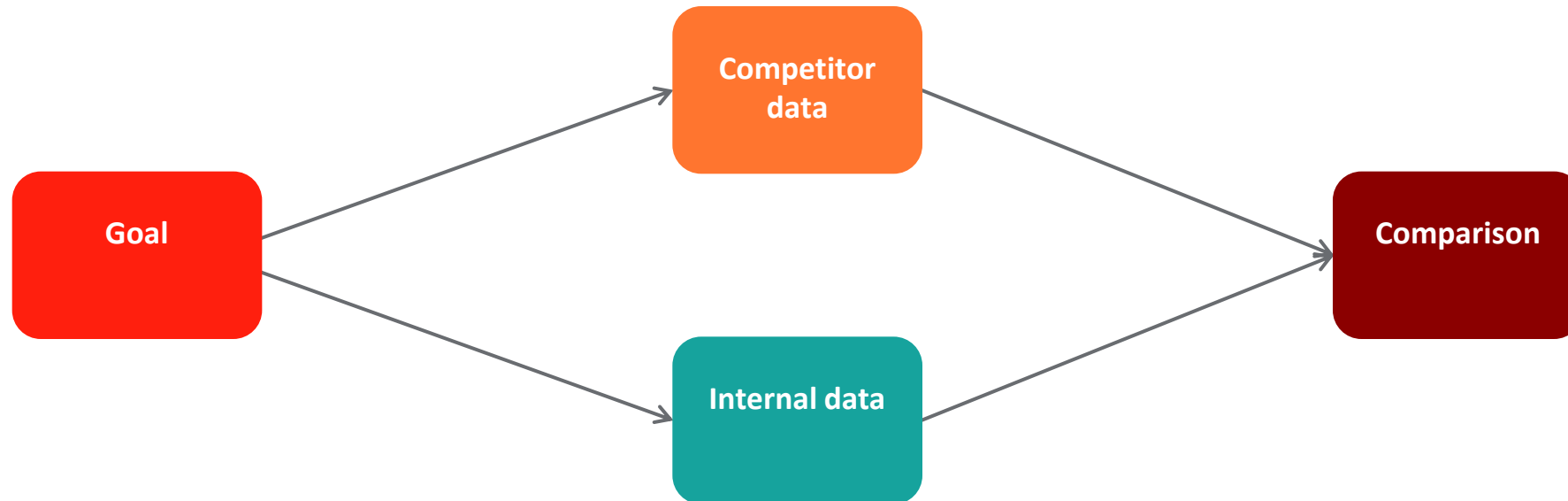
Edges

Prerequisites and data flow between tasks.

DAG constraint

Directed and acyclic: no circular dependencies that can deadlock.

A DAG unlocks safe scheduling and real parallelism



Independent branches can run concurrently; downstream work waits for explicit prerequisites.

Topological ordering is the executor's scheduling primitive

```
def visit(task_id, stack):  
    if task_id in stack:  
        raise ValueError("Cycle detected")  
    if task_id in visited:  
        return  
    stack.add(task_id)  
    for dep in tasks[task_id].depends_on:  
        visit(dep, stack)  
    stack.remove(task_id)  
    visited.add(task_id)  
    order.append(task_id)
```

What it guarantees

- Every dependency runs first.
- Cycles fail before execution.
- Independent tasks may share the same scheduling layer.

Decomposition-first

PLAN ONCE

- Generate the complete task graph up front, then execute it.
- Predictable, auditable, and easy to show a human before tools run.
- Representative patterns: HuggingGPT and Least-to-Most prompting.
- Weakness: early planning mistakes propagate through the committed plan.

Dynamic decomposition

PLAN - ACT - OBSERVE - REPLAN

- Propose the next sub-task, execute it, observe the result, then plan again.
- Failures and new information can reshape the remaining work.
- TDAG can create specialized agents for individual sub-tasks.
- Trade-off: more model calls and a harder trace.

Choose the simplest planner that can survive the task's uncertainty.

Decomposition-first

Low cost - easy to inspect - brittle when early assumptions are wrong

Dynamic

Adaptive - resilient to surprises - higher orchestration and tracing cost

Fixed vs. dynamic

Decomposition quality is still the bottleneck

34%

step-level category recall from
standard LLM decomposition

CompSkillBench

300 compositional queries across 2,200+ real MCP server tools.

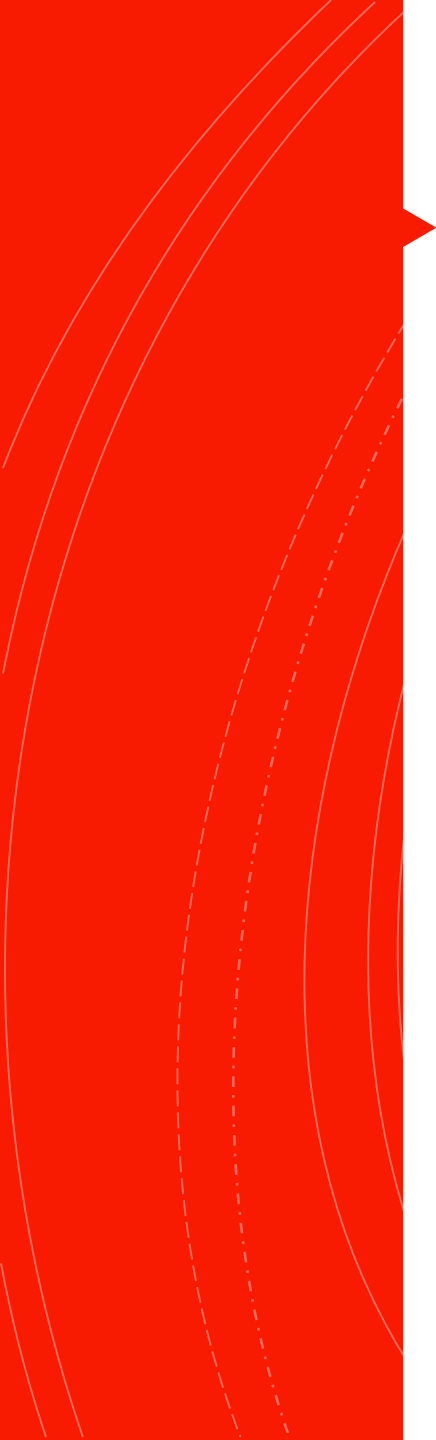
Implication

Decomposition deserves its own eval; do not treat it as a solved preprocessing step.

PART 2

Planning Algorithms

How many candidate plans should the agent consider?



Plan-and-Solve

- A single structured model call with two explicit phases.
- Designed to reduce missing-step errors in zero-shot Chain-of-Thought.
- Conceptual bridge to Plan-and-Execute agent architectures.
- One plan, generated once, executed in order - no branching.

A small prompt change creates explicit planning

Instead of

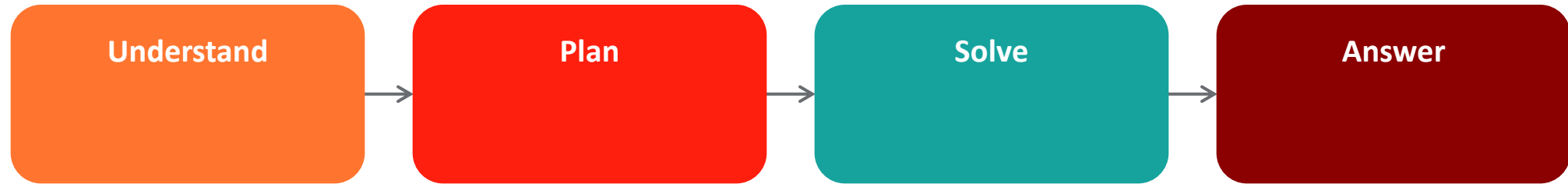
Let's think step by step.

Use

Let's first understand the problem and devise a plan. Then, let's carry out the plan step by step.

PS+ adds explicit instructions to extract variables and check calculations and common sense.

Plan-and-Solve creates two explicit phases



The plan becomes scaffolding for the execution steps.

Plan-and-Solve is lightweight - but still single-plan

Helps with

Missing steps - some calculation errors - clearer execution order

Does not solve

Semantic misunderstanding - wrong plans - lack of backtracking

It is also sensitive to the exact trigger wording: useful prompt craft, not a robust search algorithm.

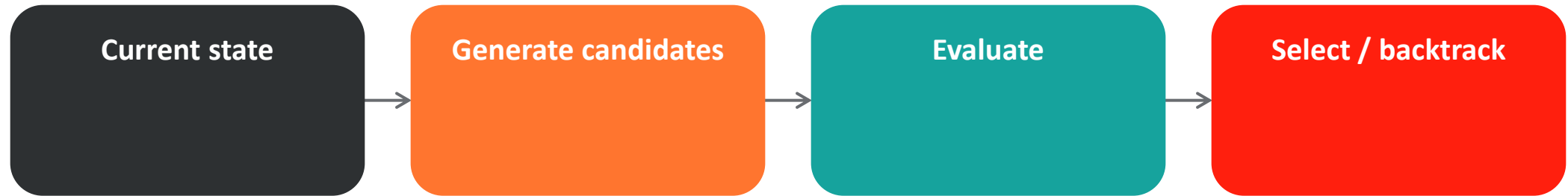
The background of the slide features a series of thin, curved lines in light gray and white, creating a sense of motion and depth. These lines are more prominent on the left side and fade towards the right.

Tree of Thoughts

EXPLORE BEFORE YOU COMMIT

- Generalizes a single reasoning chain into an explicit search tree.
- Generate several candidate next thoughts from each partial solution.
- Evaluate candidates, keep promising branches, and prune or backtrack.
- Best when lookahead matters more than the cost of exploration.

Tree of Thoughts repeats generate -> evaluate -> search



Each surviving state becomes the root of the next expansion.

BFS and DFS trade breadth for memory and backtracking

BFS

Keep the best b candidates at every depth.
Broad, shallow exploration; several
branches remain alive.

DFS

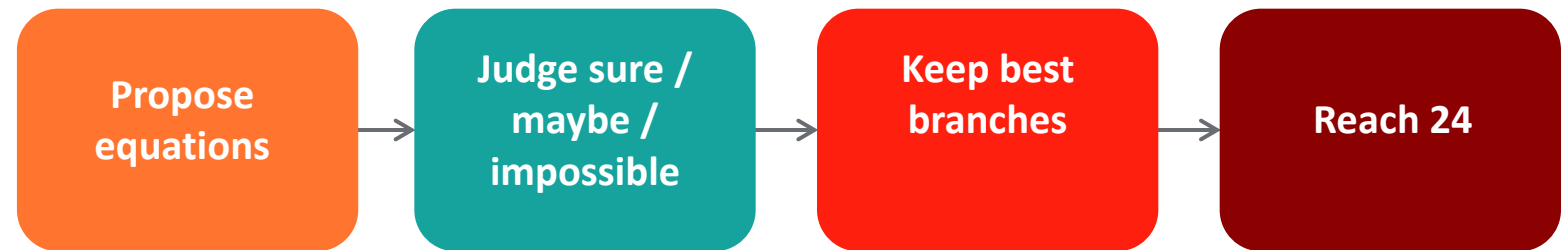
Follow one branch until a threshold or
dead end, then backtrack. Lower
concurrent breadth.

Search policy should match the shape and cost of the problem.

Game of 24 rewards lookahead

**4
numbers**

+ - x /



The LLM proposes intermediate equations; classical search decides what to expand.

The ToT-BFS loop is "generate many, score, keep the best"

```
frontier = [root]
for _ in range(depth):
    candidates = []
    for node in frontier:
        children = propose_thoughts(node, n)
        for child in children:
            child.score = evaluate(child)
            candidates.extend(children)
    frontier = sorted(
        candidates,
        key=lambda n: n.score,
        reverse=True
   )[:beam_width]
```

- Generation expands the frontier.
- Evaluation creates a search signal.
- Beam width controls cost and breadth.

Plan-and-Solve and Tree of Thoughts serve different tasks

	Plan-and-Solve	Tree of Thoughts
Plans considered	1	Many branches
Mechanism	Structured prompt	Generate -> evaluate -> search
Backtracking	None	Yes
Relative cost	about 1x CoT call	Can exceed 100x
Best fit	Mostly linear tasks	Lookahead and exploration

Exploration buys quality with compute.

Evidence

Mini Crosswords: roughly 60% word-level success with DFS + pruning, above CoT baselines.

Cost

Game of 24 visited ~70 nodes and sampled each three times - often 100x+ a single CoT pass.

Search breadth

MCTS + ENVIRONMENT FEEDBACK

- LATS combines an LLM with Monte Carlo Tree Search over action trajectories.
- Reasoning, acting, planning, and reflection happen inside one search loop.
- Each branch interacts with a real environment or simulator.
- External rewards guide the search instead of relying only on model self-judgment.

The LLM plays three roles inside LATs

Action generator

Proposes candidate next actions or reasoning steps from the trajectory so far.

Value function

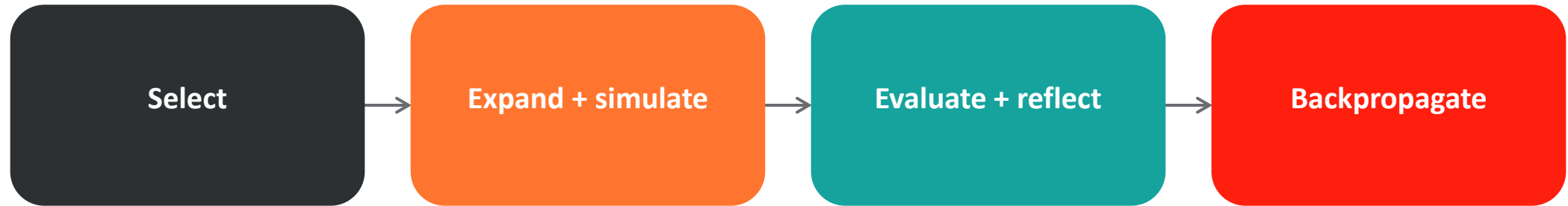
Estimates future reward so MCTS knows where to spend its search budget.

Reflection mechanism

Explains why a branch failed and feeds that lesson into later expansions.

Generation explores. Value prioritizes. Reflection carries learning forward.

MCTS turns environment outcomes into better search choices



Selection follows aggregate value; backpropagation updates every ancestor on the sampled path.

External feedback is the defining difference from ToT

Tree of Thoughts

The model judges partial reasoning as sure, maybe, or impossible.

LATS

The environment reports what happened: tests pass, an API succeeds, or a page contains the target.

LATS spends extra compute on grounded evidence, not simply more self-opinion.

A LATS iteration expands, checks, reflects, and learns

```
for _ in range(iterations):
    leaf = select(root)
    children = expand_and_simulate(
        leaf, llm, env, n_actions
    )
    for child in children:
        feedback = env.external_feedback(child.state)
        if feedback.failed:
            child.reflections.append(
                llm.reflect_on_failure(child.state, feedback)
            )
        backpropagate(child, feedback.score)
```

- env.step() executes each candidate.
- Reward replaces pure self-evaluation.
- Failure reflections steer later samples.
- Backpropagation updates ancestor values.

Planning methods form a cost-to-grounding spectrum

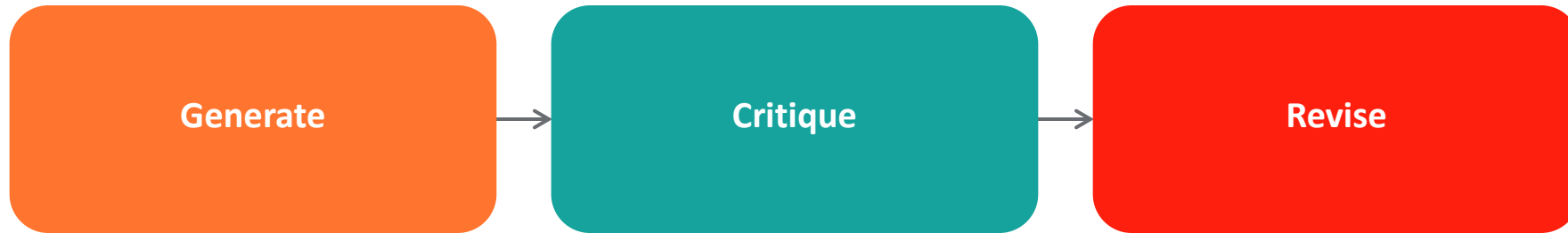
	Plan-and-Solve	Tree of Thoughts	LATS
Branches	1	Many	Many, MCTS-guided
Evaluation	None	LLM self-evaluation	Environment + value function
Learning	No	Within one search	Reflections steer later search
Relative cost	Lowest	High	Highest
Best fit	Linear work	Lookahead without checks	Checkable environments

PART 5

Self-Correction & Reflection

Checking whether execution produced something good

Reflection closes a loop around a complete output



The loop can wrap any generator: a direct prompt, Plan-and-Solve, or Tree of Thoughts.

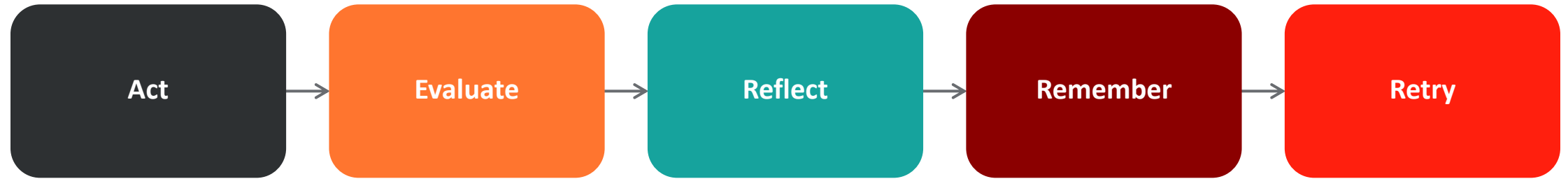
A large red speech bubble graphic with a white outline, pointing downwards. The word "Reflexion" is written in white text inside the bubble.

Reflexion

VERBAL REINFORCEMENT LEARNING

- Self-Refine revises one draft; Reflexion retries the entire task across trials.
- An environment or task-specific check provides pass/fail or scalar reward.
- The agent writes a verbal lesson after failure and stores it in episodic memory.
- The next trial sees recent reflections in context - no fine-tuning required.

Reflexion carries lessons across complete task attempts



A compact episodic buffer makes prior failures visible to the next trial.

The retry may contain its own ReAct loop; Reflexion wraps the whole episode.

The memory buffer turns failure into context for the next trial

```
memory = []
for trial in range(max_trials):
    trajectory = llm.act(
        task,
        past_reflections=memory[-memory_size:]
    )
    outcome = env.evaluate(trajectory)
    if outcome.success:
        return trajectory
    memory.append(
        llm.reflect(task, trajectory, outcome)
    )
```

- Memory is capped to recent lessons.
- Feedback can be binary or scalar.
- Success ends the loop immediately.
- Exhausted retries surface to a caller or human.

Reflection needs a stopping rule and a trustworthy reward.

Stopping criteria

Cap consecutive failed attempts. Reflexion experiments used a small limit rather than retrying forever.

Grounded reward

Prefer executed tests, game outcomes, validators, or real tool results over a verbal guess.

Bound the loop

The same reflection idea operates at three different scopes

Self-Refine

Revise one finished draft inside a single episode.

Reflexion

Retry the whole task and carry lessons across episodes.

LATS

Reflect on failed branches inside an MCTS trajectory search.

One draft. One task. One search branch.

A critique only works when the rubric exists first

Accuracy

Are claims correct and grounded?

Consistency

Do steps agree with each other?

Completeness

Were required parts omitted?

Adherence

Did the output follow the original instructions?

A separate critic helps break shared blind spots

Self-reflection

Same model and context generate and judge.

Cheap and simple, but prone to self-validation.

Separate critic

Different context, rubric, or model. More orchestration, but a stronger independent check.

Separation changes the critic's information and incentives - not just its label.

Why self-critique fails

- A model grading its own reasoning often agrees with itself.
- A fluent critique can misidentify the real defect while sounding convincing.
- Without an external verification signal, no genuinely new information enters the loop.
- Process reward models score intermediate steps, but they are still a learned signal.

Grounded reflection introduces information from outside the model

```
# Ungrounded  
critique = llm("Critique this code")  
revision = llm(code + critique)
```

Opinion about the draft

```
# Grounded  
result = test_fn(code)  
if not result.passed:  
    revision = llm(code +  
    result.output)
```

A failing test from reality

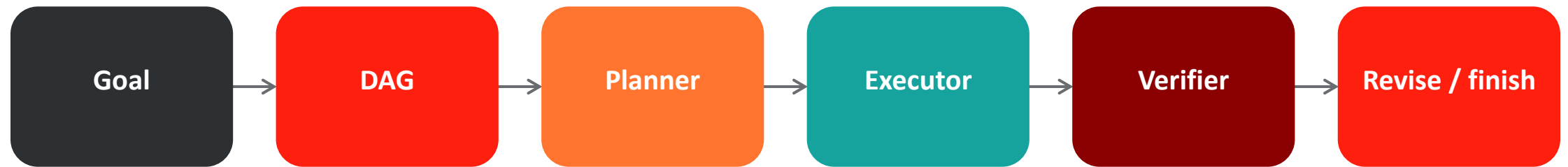
External signals add information the generator cannot dismiss.

If you can run a test, run the test.

- Use execution for code and SQL.
- Use schema validation for structured output.
- Use retrieval or source checks for factual claims.
- Reserve pure LLM critique for tone, clarity, and other soft judgments.

Run the test

A robust agent connects the full control loop



**Dependencies control order. Search controls alternatives.
Verification controls trust.**

Choose the algorithm by uncertainty, stakes, and cost

Mostly linear

Decomposition-first +
Plan-and-Solve

Surprises likely

Dynamic decomposition +
replanning

Lookahead

Tree of Thoughts when
self-evaluation is enough

Checkable world

LATS when branches can
receive real rewards

Use the least expensive structure that can detect and recover from the failures you actually expect.

The architecture is a sequence of explicit choices

Decompose

Create a valid DAG; evaluate decomposition quality.

Plan

Use PS for linear work, ToT for lookahead, or LATS for grounded search.

Reflect

Use Self-Refine for a draft, Reflexion for retries, and external feedback whenever possible.